RV College of Engineering®
Mysore Road, RV Vidyaniketan Post,
Bengaluru - 560059, Karnataka, India

QUEUES

FIFO concept • Queue ADT • Operations • Types

Simple • Circular • Double-Ended • Priority Queues

Data Structures and Applications

Based on E. Horowitz, S. Sahni and S. Anderson-Freed
Fundamentals of Data Structures in C

Learning Objectives

Define a queue and explain the First-In-First-Out (FIFO) principle.

Describe the Queue Abstract Data Type (ADT) and its basic operations.

Trace enqueue, dequeue, front and rear operations using an array representation.

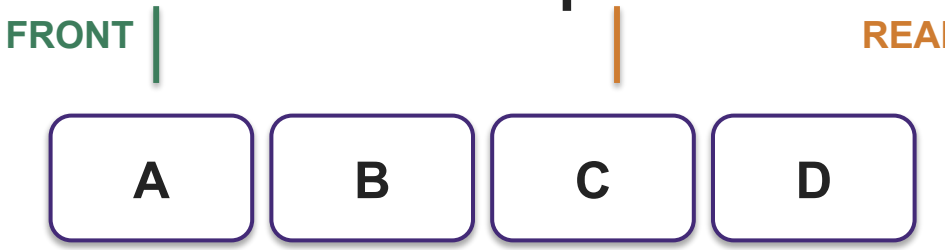
Explain the limitations of a simple linear queue and the need for a circular queue.

Differentiate circular queues, double-ended queues (deque) and priority queues.

Relate queue structures to common computing applications.

Queue: Basic Concept

A queue is an ordered list in which insertions take place at one end and deletions take place at the opposite end.



Deletion occurs at FRONT ← A B C D → Insertion occurs at REAR

Key idea

Only the two ends are used for the standard queue operations.

FIFO: First-In-First-Out

The element that enters first is the element that leaves first.



Enqueue: add the new item at the rear.

Dequeue: remove the item at the front.

Thus, the ordering of service follows arrival order.

Queue as an Abstract Data Type (ADT)

The ADT specifies what a queue does, independent of how the queue is represented or implemented.

CREATEQ	Create an empty queue with a specified maximum size.
ISEMPTYQ	Determine whether the queue contains no elements.
ISFULLQ	Determine whether the queue has reached its capacity.
ADDQ / ENQUEUE	Insert an item at the rear of the queue.
DELETEQ / DEQUEUE	Remove and return the item at the front.

Textbook terminology: Horowitz et al. use AddQ/DeleteQ; these correspond to enqueue/dequeue.

Queue Operations: Enqueue and Dequeue

ENQUEUE (ADDQ)

Adds a new element at the REAR.

1. Check whether the queue is full.
2. Advance rear.
3. Store the new item at queue[rear].

Effect: the queue grows at the rear.

DEQUEUE (DELETEQ)

Removes the element at the FRONT.

1. Check whether the queue is empty.
2. Advance front.
3. Return the removed item.

Effect: the queue shrinks at the front.

Error conditions: Queue Full during insertion; Queue Empty during deletion.

Front and Rear

Front and rear identify the two ends of the queue.



FRONT

Points to the element that will be removed next.

REAR

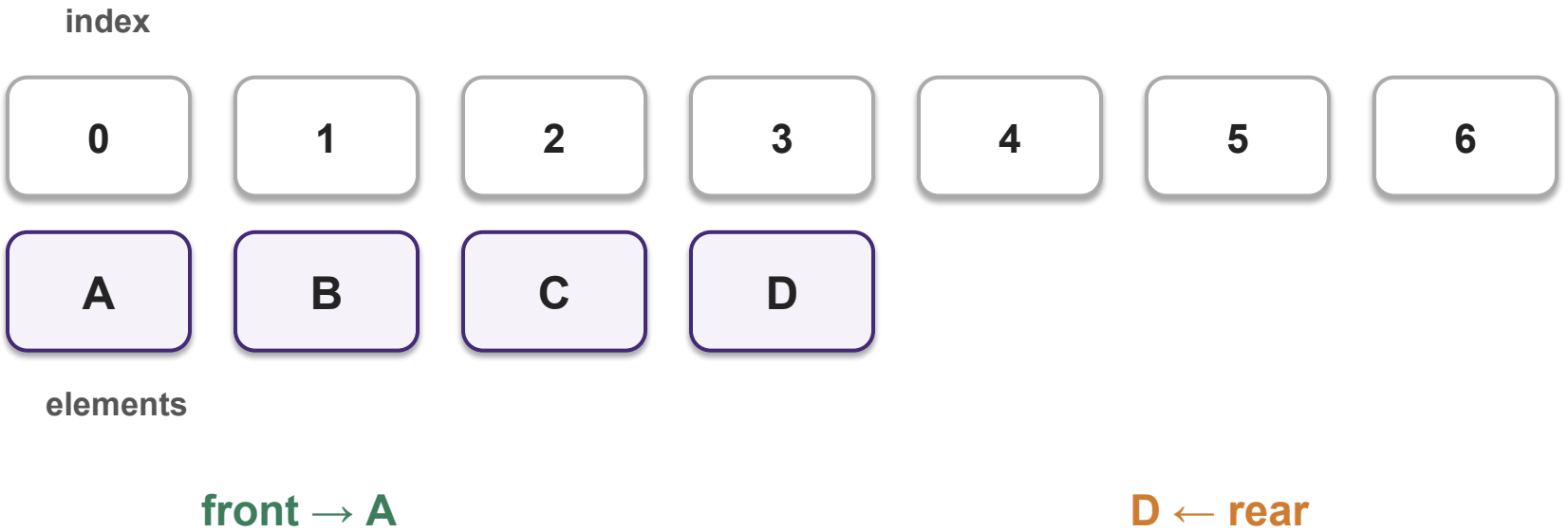
Marks the position where the next element will be inserted.

ACCESS

Standard queue access is restricted to the two ends.

Array Representation of a Simple Queue

A one-dimensional array can store the queue, with front and rear maintaining its boundaries.

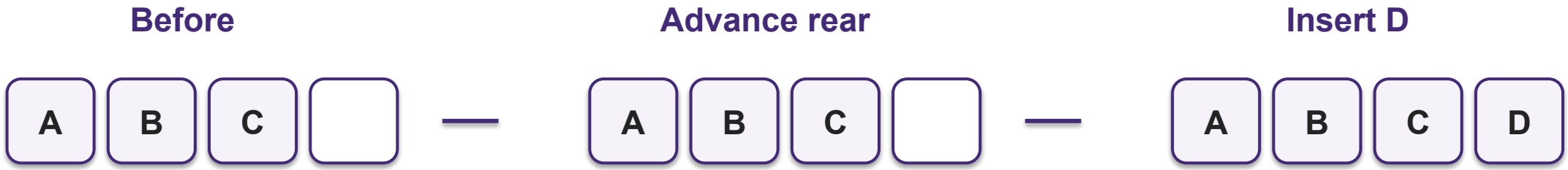


Sequential storage is simple and efficient for basic operations.

However, a linear array queue can waste space after deletions.

Enqueue: Step-by-Step Trace

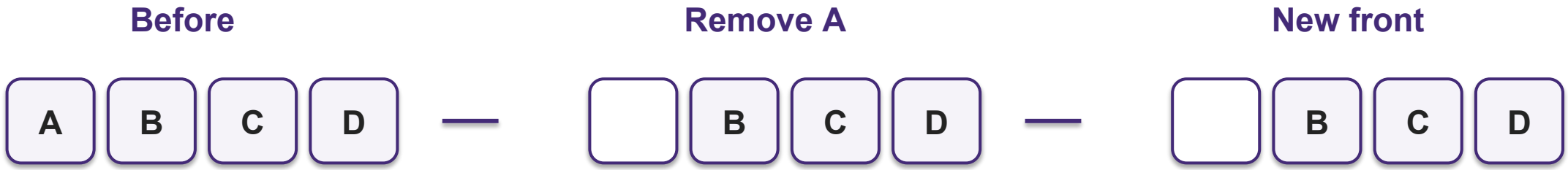
Suppose the queue contains A, B, C and we enqueue D.



The new item is always placed at the rear; existing elements keep their relative order.

Dequeue: Step-by-Step Trace

Starting with A, B, C, D, dequeue removes A and advances the front.



The removed item is the front element. The next element becomes the new front.

Simple (Linear) Queue

Definition

The simplest queue representation. Insertions occur at the rear and deletions occur at the front.

It follows FIFO ordering.

Advantages

- Easy to understand and implement
- Direct mapping to an array
- Enqueue and dequeue can be constant-time when pointers are maintained

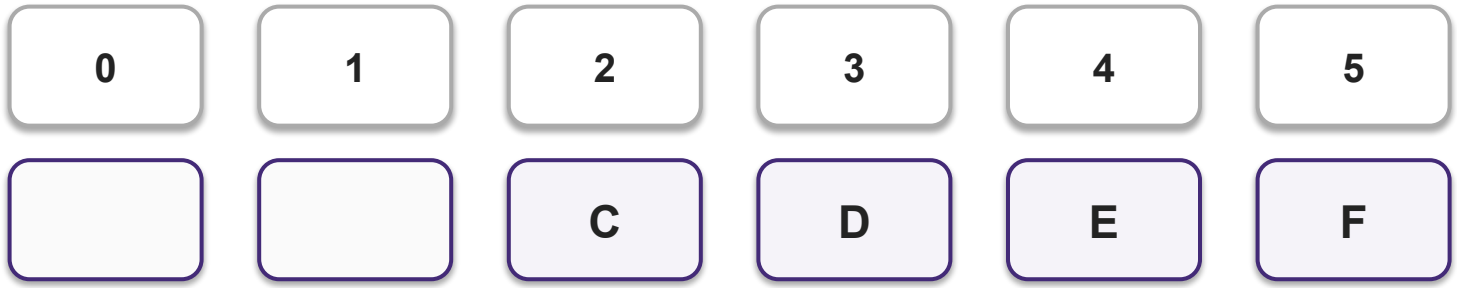
Limitation

After repeated deletions, unused cells can appear at the beginning of the array while rear reaches the last index.

This wasted-space condition motivates the circular queue.

Problem with the Linear Queue

Example: a queue of size 6 after several deletions and insertions.

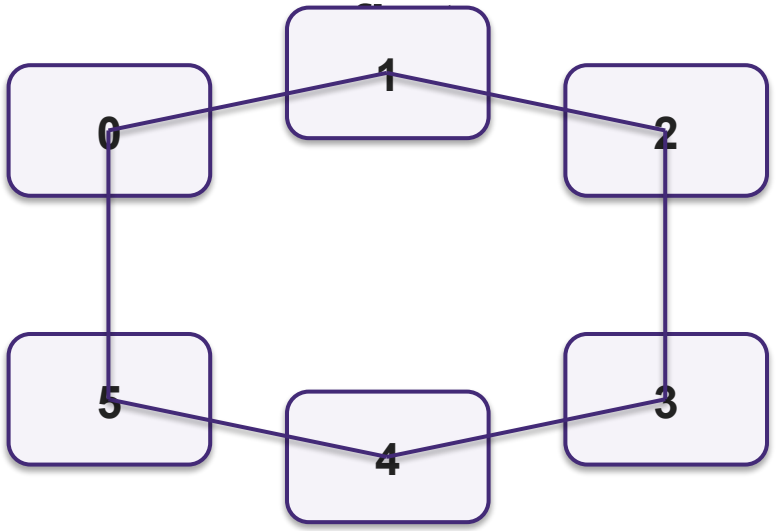


The cells at the beginning are free, but a simple linear implementation may still report the queue as full when rear reaches the last position.

Shifting elements to reclaim space can make an insertion or deletion O(n).

Circular Queue

A circular queue treats the array as if the last position is connected back to the



When rear reaches the last index, the next position can wrap to index 0 if that position is free.

Typical wrap-around rule: next position = (position + 1) mod MAX_QUEUE_SIZE

Circular Queue: Enqueue and Dequeue

Circular indexing reuses positions released by earlier dequeue operations.

ENQUEUE IN CIRCULAR QUEUE

1. Check whether the queue is full.
2. Move rear to the next circular position.
3. Insert the new item there.
4. Continue from index 0 after the last index.

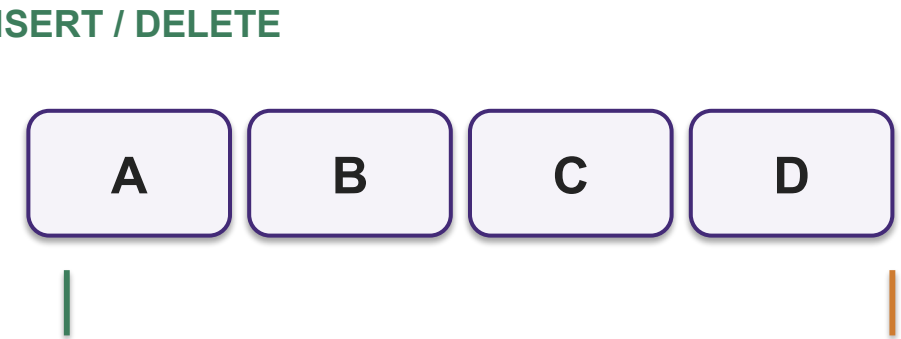
DEQUEUE IN CIRCULAR QUEUE

1. Check whether the queue is empty.
2. Remove the item at front.
3. Move front to the next circular position.
4. Freed cells can later be reused.

Main benefit: avoids unnecessary shifting and makes better use of a fixed-size array.

Double-Ended Queue (Deque)

A deque allows insertion and deletion at both ends of the queue.



Input-Restricted Deque

Insertion is permitted at only one end; deletion is permitted at both ends.

Output-Restricted Deque

Deletion is permitted at only one end; insertion is permitted at both ends.

Priority Queue

In a priority queue, removal is determined by priority rather than strictly by arrival time.

Job A	Job B	Job C	Job D
Priority 3	Priority 1	Priority 2	Priority 1

If lower numerical value means higher priority, Job B and Job D are served before Job C and Job A.

Typical use

CPU/job scheduling and service systems.

Key distinction

Priority can override arrival order.

Comparing Queue Types

Type	Insertion	Deletion	Main characteristic
Simple / Linear	Rear	Front	Strict FIFO; possible wasted space
Circular	Rear	Front	Wrap-around reuses array positions
Deque	Front or Rear	Front or Rear	Both ends are accessible
Priority	Based on implementation	Highest/lowest priority	Service order depends on priority

Queue Applications

Job scheduling

Jobs wait for processor service.

Printer spooling

Print requests are handled in an ordered waiting line.

I/O buffering

Data waits between producers and consumers.

Network packets

Packets can be buffered before transmission/service.

Simulation event lists

Events can be maintained in time/order-based queues.

Breadth-First Search

A queue supports level-by-level graph traversal.

Queue Operation Complexity

With front and rear maintained appropriately, basic queue operations can be performed efficiently.

Operation	Typical array queue	Purpose
Enqueue	**O(1)**	Insert at rear
Dequeue	**O(1)**	Remove from front
Front / Peek	**O(1)**	Inspect next item
Rear	**O(1)**	Inspect most recently inserted item

Caution: shifting elements in a linear array queue may require O(n) movement.

Worked Example: Trace the Queue

Perform: ENQUEUE(10), ENQUEUE(20), ENQUEUE(30), DEQUEUE(), ENQUEUE(40).

1	enqueue 10	10
2	enqueue 20	10 20
3	enqueue 30	10 20 30
4	dequeue → 10	20 30
5	enqueue 40	20 30 40

Final queue: 20, 30, 40 | Front = 20 | Rear = 40

Common Errors and Edge Conditions

QUEUE UNDERFLOW

Attempting dequeue when the queue is empty.

Response: report Queue Empty / underflow and do not remove an element.

QUEUE OVERFLOW

Attempting enqueue when a fixed-size queue is full.

Response: report Queue Full / overflow and do not insert the item.

WRAP-AROUND

In a circular queue, update indices modulo the queue capacity so that freed positions can be reused.

Carefully define the empty and full conditions for the chosen circular-queue representation.